



Laboratorio No. 1 Semana #13 – APIS con swagger & postman

Nombre: Fátima Joanna López Quiñonez **Carné:** 1088825

Objetivo: Comprender y consumir una API REST mediante Swagger UI y Postman, identificando endpoints, métodos HTTP, parámetros, cuerpos JSON, códigos de respuesta y operaciones CRUD. El estudiante deberá interpretar la documentación de una API, ejecutar solicitudes y resolver un reto práctico utilizando evidencia técnica.

Investigación previa:

1. ¿Qué es una API REST?

Es una forma de comunicar dos sistemas por medio de HTTP, donde todo se trata como un recurso que se manipula con métodos estándar. Cada petición es independiente y lleva toda la información necesaria.

2. ¿Qué es un endpoint?

Es un punto de acceso específico de una API para realizar una operación sobre un recurso.

3. ¿Qué diferencia existe entre una URL y un endpoint?

La URL es la dirección completa de un recurso en la web. El endpoint es esa misma ruta junto con el método HTTP, es decir, con la operación disponible allí.

4. ¿Qué función cumplen los métodos HTTP GET, POST, PUT/PATCH y DELETE?

GET consulta datos, POST crea un recurso, PUT lo reemplaza completo, PATCH lo modifica parcialmente y DELETE lo elimina.

5. ¿Qué es JSON y por qué es común en las APIs?

Es un formato de texto ligero que organiza datos en pares clave-valor, como {"nombre": "Ana"}. Es común en las APIs porque es fácil de leer, liviano y compatible con casi todos los lenguajes.

6. ¿Qué es un código de estado HTTP? Mencione ejemplos de 200, 404 y 500.

Es un número que indica el resultado de la petición. Ejemplos: 200 (éxito), 404 (recurso no encontrado) y 500 (error interno del servidor).

7. ¿Qué información se puede encontrar en la respuesta de una API?

Incluye el código de estado, los encabezados y el cuerpo con los datos, normalmente en JSON. También puede traer mensajes de error si algo falló.

8. ¿Qué ventajas tiene utilizar Postman para probar una API antes de programarla?

Permite probar endpoints sin escribir código, lo que ayuda a detectar errores antes de programar. Además, se pueden guardar las peticiones y reutilizarlas.

9. ¿Qué es y para qué sirve Swagger UI?

Es una interfaz web interactiva que muestra la documentación de una API. Permite ver los endpoints y probarlos directamente desde el navegador.

10. ¿Qué función cumple Postman en el desarrollo y prueba de APIs?

Es una herramienta para diseñar, probar y documentar APIs enviando peticiones y analizando las respuestas. También permite automatizar pruebas y compartirlas con el equipo.

1. Preparación del entorno

Instalar o utilizar Postman: <https://www.postman.com/downloads/>

API a explorar, Swagger Petstore 3: <https://petstore3.swagger.io/>

Importante: La actividad se realizará mediante solicitudes HTTP GET y análisis de las respuestas JSON. No se requiere API Key para esta práctica.

2. Exploración inicial

Ingrese a Swagger Petstore y localice las operaciones relacionadas con /pet



- Identifique un endpoint que permita consultar una mascota por ID.
- Indique el método HTTP utilizado.
- Indique qué parámetro necesita la solicitud.
- Ejecute la solicitud con Try it out y registre el código HTTP recibido.
- Explique con sus palabras qué información contiene la respuesta.

Evidencia: captura de Swagger mostrando la operación y su respuesta.

The screenshot shows the Swagger UI interface. At the top, there's a table with 'Name' and 'Description'. Below it, a parameter 'petid' is defined as 'ID of pet to return' with a type of 'integer (\$int64)' and a path location. A text input field contains the value '12345'. Below the input are 'Execute' and 'Clear' buttons. Under the 'Responses' section, the 'Curl' tab shows a command: `curl -X 'GET' \ 'https://petstore3.swagger.io/api/v3/pet/12345' \ -H 'accept: application/json'`. The 'Request URL' tab shows the URL: `https://petstore3.swagger.io/api/v3/pet/12345`. The 'Server response' section shows a '200' status code. The 'Response body' tab displays a JSON object: `{ "id": 12345, "category": { "id": 1, "name": "dogs" }, "name": "fluffy", "photoUrls": [], "tags": [ { "id": 0, "name": "tag1" }, { "id": 1, "name": "tag2" } ], "status": "available" }`. A 'Download' button is visible at the bottom right of the response body.

Explicación: Se utilizó el método GET, debido a que se desea consultar información de una mascota existente sin realizar modificaciones. El endpoint necesita de un parámetro de petId, debido a que es el índice que requiere el sistema para saber de qué mascota traer la información. La respuesta contiene información de la mascota en formato JSON incluyendo su id, categoría, nombre, etiquetas y estado. El código HTTP recibido fue el 200, indicado que la consulta fue procesada exitosamente.

3. Crear un recurso con POST

Utilice POST /pet para crear una mascota. Construya un JSON propio; no copie exactamente el ejemplo de Swagger.

El JSON debe incluir, como mínimo:

id
name
status
category
tags

Registre el ID utilizado y verifique la respuesta. Luego consulte nuevamente la mascota por medio de GET /pet/{petId}.

Evidencia: JSON enviado + respuesta de creación + consulta posterior.



Request body required application/json

Create a new pet in the store

Edit Value | Schema

```
{
  "id": 81,
  "name": "Oscar Piastri",
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 2819,
      "name": "string"
    }
  ],
  "status": "available"
}
```

Execute Clear

A) Body con la información de la nueva mascota

Responses

Curl

```
curl -X 'POST' \
  'https://petstore3.swagger.io/api/v3/pet' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": 81,
    "name": "Oscar Piastri",
    "category": {
      "id": 1,
      "name": "Dogs"
    },
    "photoUrls": [
      "string"
    ],
    "tags": [
      {
        "id": 2819,
        "name": "string"
      }
    ],
    "status": "available"
  }'
```

Request URL

https://petstore3.swagger.io/api/v3/pet

Server response

Code	Details
200	Response body <pre>{ "id": 81, "category": { "id": 1, "name": "Dogs" }, "name": "Oscar Piastri", "photoUrls": ["string"], "tags": [{ "id": 2819, "name": "string" }], "status": "available" }</pre> Download

B) Mensaje de operación exitosa (código HTTP: 200) y el body de la mascota nueva creada



Name	Description
petid * required integer(\$int64) (path)	ID of pet to return 81
Execute Clear	
Responses	
Curl <pre>curl -X 'GET' \ 'https://petstore3.swagger.io/api/v3/pet/81' \ -H 'accept: application/json'</pre>	
Request URL <pre>https://petstore3.swagger.io/api/v3/pet/81</pre>	
Server response	
Code	Details
200	Response body <pre>{ "id": 81, "category": { "id": 1, "name": "Dogs" }, "name": "Oscar Piastri", "photoUrls": ["string"], "tags": [{ "id": 2819, "name": "string" }], "status": "available" }</pre> Download

C) Usando el método get, se confirmó que la creación fue exitosa.

4. Modificar el recurso

- Modifique la mascota creada utilizando PUT /pet. Cambie al menos dos atributos, por ejemplo nombre y estado.
- ¿Qué método HTTP utilizó y por qué?

Se utilizó el método PUT porque es el que permite actualizar los datos de un recurso existente, exceptuando en este caso el id porque sino el programa no identifica que mascota se quiere actualizar

- ¿Qué datos fueron enviados en el body? (Los datos resaltados son los modificados)

```
{
  "id": 81,
  "category": {
    "id": 1,
    "name": "Orange cat"
  },
  "name": "Oscar Piastri",
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 2023,
      "name": "pastry"
    }
  ],
  "status": "available"
}
```

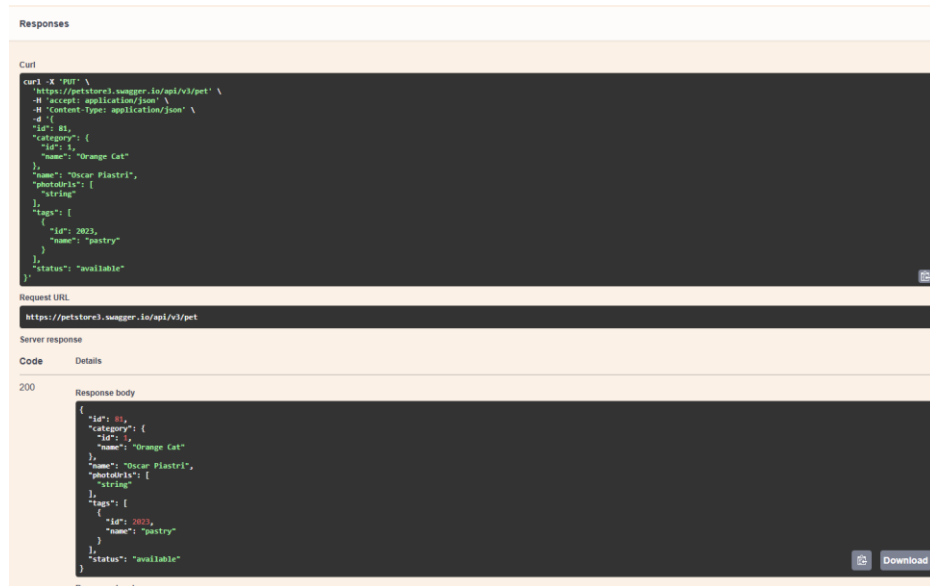


- ¿Qué código HTTP devolvió la API?

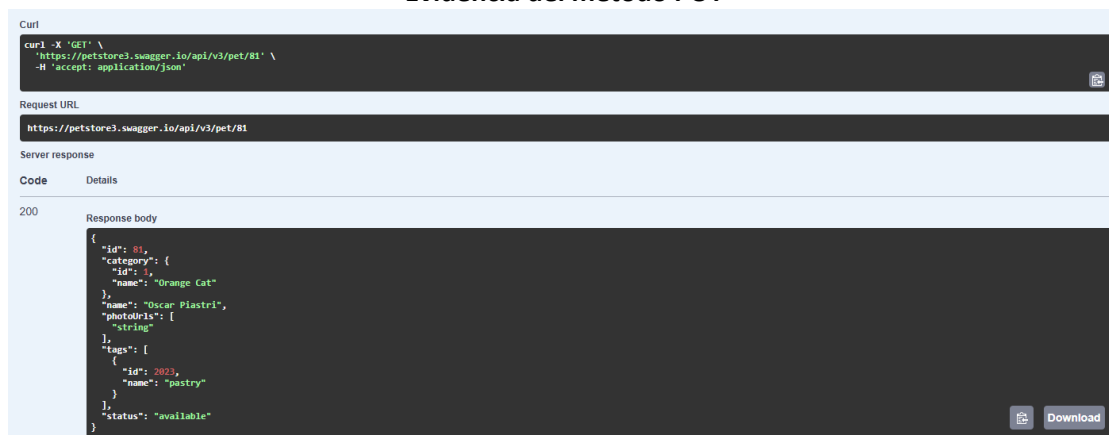
200, confirmando que la operación fue exitosa

- ¿Cómo comprobó que el cambio realmente ocurrió?

Utilizando el método GET y el ID de la mascota modificada. La respuesta mostró el nuevo nombre de categoría y la etiqueta modificada.



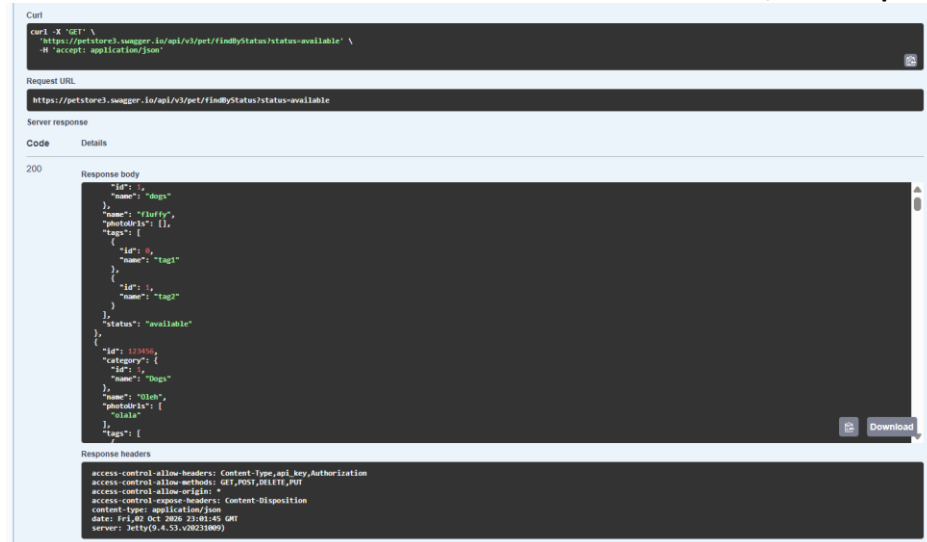
Evidencia del método PUT



Evidencia del método GET

5. Consultar utilizando parámetros

- Utilice GET /pet/findByStatus y consulte las mascotas con estado available.
- **Identifique el parámetro utilizado.**
status
- **Explique dónde se envía ese parámetro.**
Se envía como query parameter, después del signo '?' en la URL
- Observe la estructura de la respuesta.
- Indique qué sucede si cambia el valor del parámetro por pending.
Se realiza una consulta para obtener todas las mascotas con el valor de 'pending' en el parámetro 'status'.



Evidencia de método GET {findByStatus}

6. Reproducir las operaciones en Postman

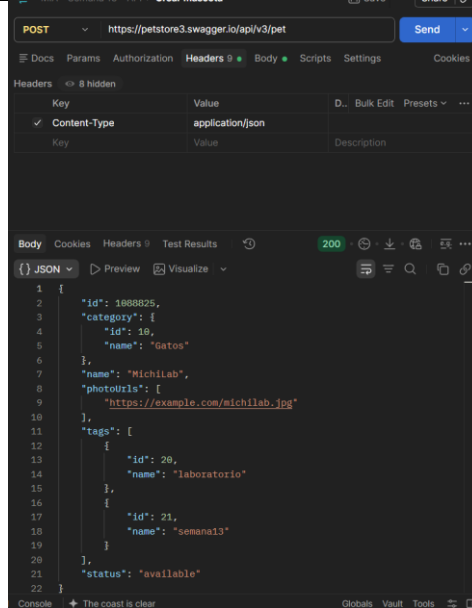
- Ahora reproduzca en Postman las operaciones realizadas en Swagger. Cree una colección llamada 'MIA - Semana 13 - API'.
- La colección debe contener, como mínimo:
- GET – Consultar mascota por ID
- POST – Crear mascota
- PUT – Modificar mascota
- GET – Buscar por status
- DELETE – Eliminar mascota
- En cada solicitud configure manualmente método, URL, parámetros, headers y body cuando corresponda. No utilice únicamente la opción de importar una colección.

Nombre	Método	Consulta
Consultar mascota por ID	GET	



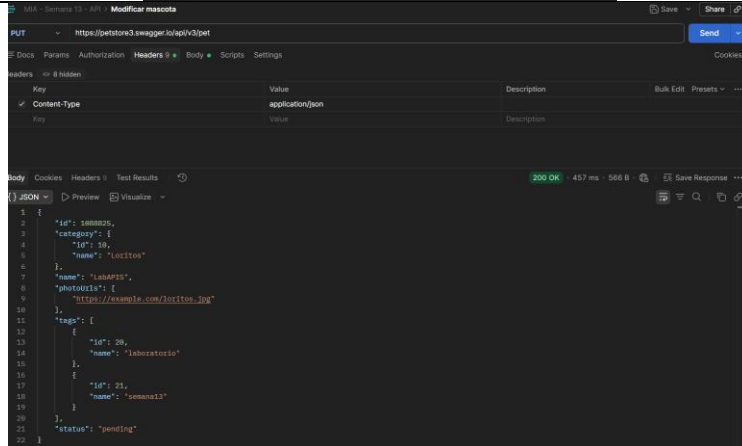
Crear mascota

POST



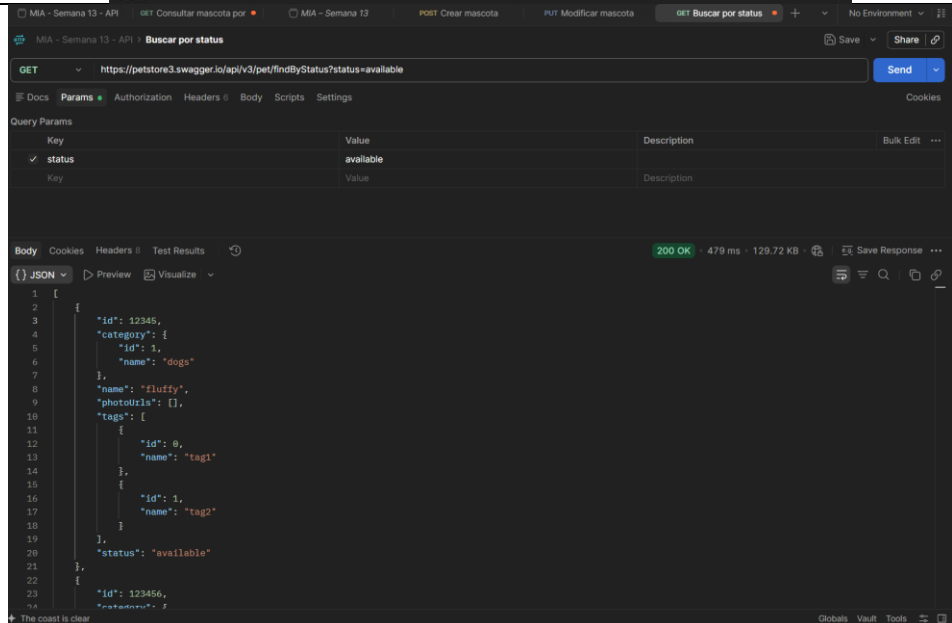
Modificar
mascota

PUT

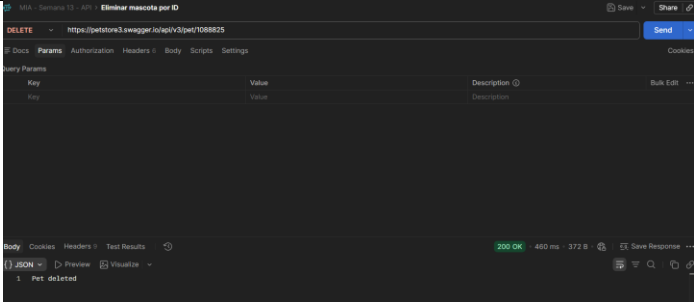
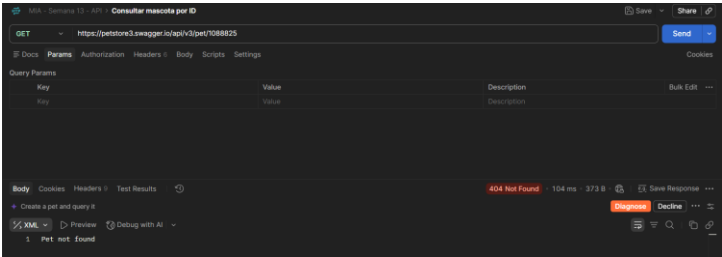


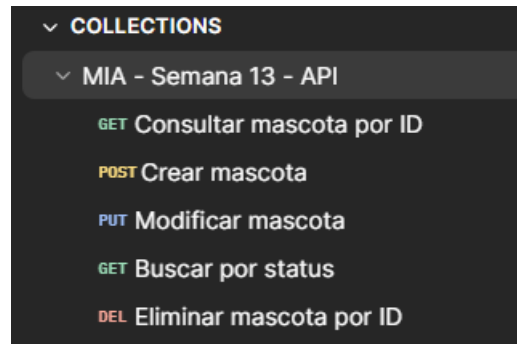
Buscar por
status

GET





Eliminar mascota	DELETE	 Evidencia de eliminación  Evidencia de confirmación de eliminación usando GET
------------------	--------	--



Evidencia de colección de request en Postman

7. Prueba de errores

Una API también debe analizarse cuando algo sale mal. Realice al menos tres pruebas que generen respuestas diferentes a una solicitud exitosa.

Ejemplos de escenarios:

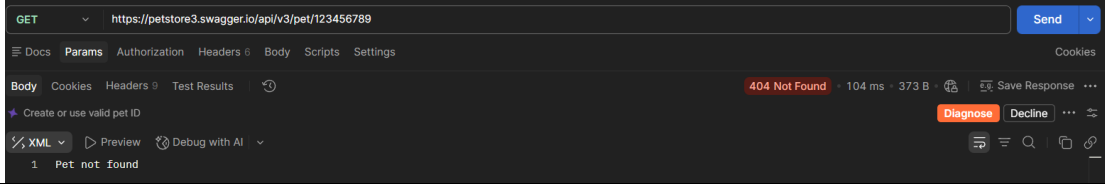
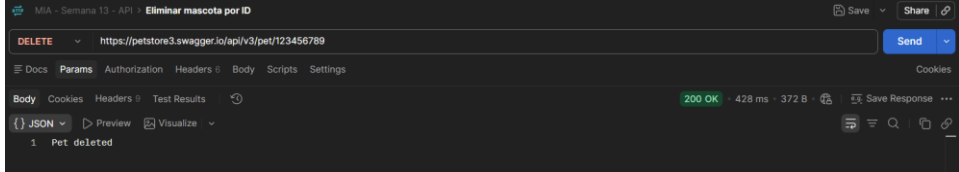
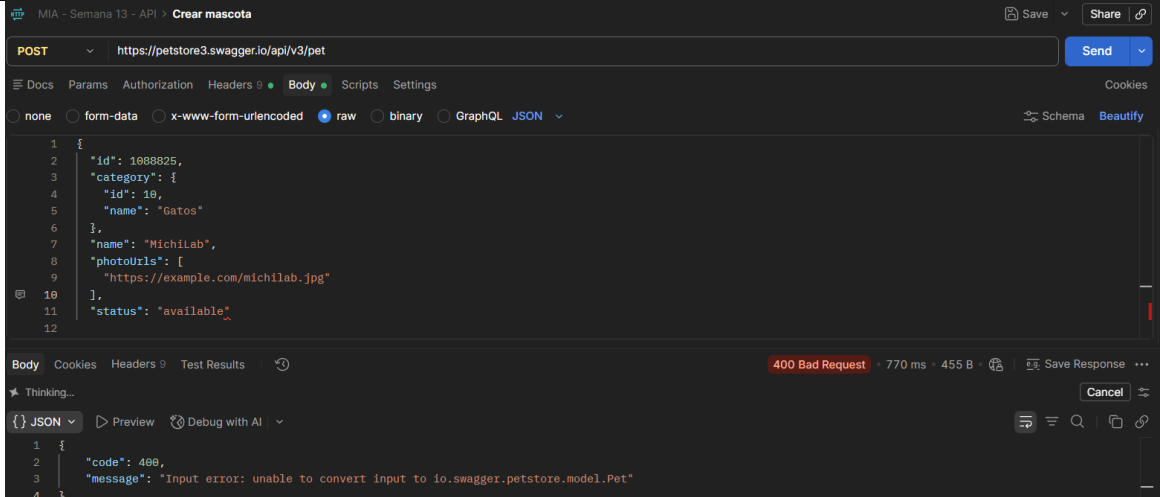
1. Consultar un ID inexistente.
2. Enviar un JSON incorrecto o incompleto.
3. Utilizar un parámetro con un valor no esperado.
4. Intentar eliminar un recurso que ya no existe.

Para cada prueba registre:

- a) Solicitud realizada.
- b) Código HTTP recibido.
- c) Respuesta de la API.
- d) Qué esperaba que ocurriera.
- e) Qué ocurrió realmente.
- f) Explicación de la causa

Caso	Solicitud realizada	Código HTTP recibido	Qué esperaba que ocurriera	Qué ocurrió realmente	Explicación de la causa
Consultar un ID inexistente	GET [URL]/pet/123456789	404 Not Found	Que la API indicara que la mascota no	La API devolvió un error de recurso no encontrado	El petId enviado no corresponde a una mascota existente



			existe		
					
Eliminar un recurso que no existe	DELETE [URL]/pet/123456789	200 OK	Que la API indicara que no se puede eliminar la mascota porque no existe	La API respondió con un mensaje que sí se había realizado la eliminación	La API Petstore implementa la operación DELETE de manera que devuelve 200 OK junto con el mensaje pet deleted, incluso cuando el ID indicado no corresponde a un recurso existente
					
Enviar un JSON incompleto	POST [URL]/pet Body: Se eliminó la parte de tags y se le quitó la llave de cierre del JSON	400 Bad Request	Que la API rechazara el JSON	Postman detectó que el body no era válido	El JSON tenía una estructura sintáctica incorrecta
					



8. API Detective

Suponga que usted recibe el siguiente requerimiento de un sistema:

“El sistema debe registrar una mascota, consultar su información, actualizar su estado y finalmente eliminarla. Si el recurso no existe, el sistema debe poder identificarlo mediante la respuesta de la API.”

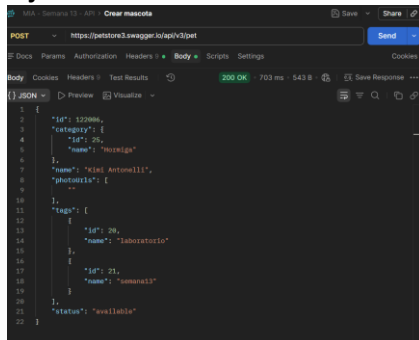
Resuelva el reto:

- Defina qué endpoint utilizaría para cada operación.
- Defina el método HTTP correspondiente.
- Indique qué respuesta debe analizar el programa.
- Ejecute el flujo completo en Postman.

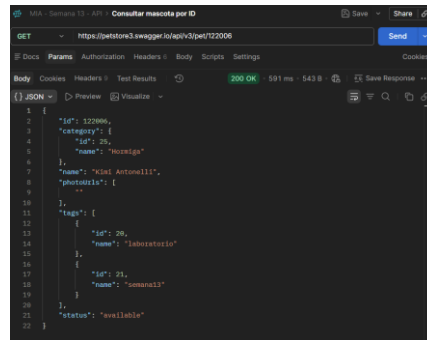
Capture evidencia de cada paso.

Operación	Endpoint	Método	Respuesta que debe analizar el programa
1. Registrar mascota	[URL]/pet	POST	Código HTTP y JSON de la mascota creada
2. Consultar mascota	[URL]/pet/122006	GET	Código HTTP y datos del recurso
3. Actualizar estado	[URL]/pet	PUT	Código HTTP y datos actualizados
4. Eliminar mascota	[URL]/pet/122006	DELETE	Código HTTP de la operación
5. Comprobar inexistencia	[URL]/pet/122006	GET	404 Not Found y mensaje de error

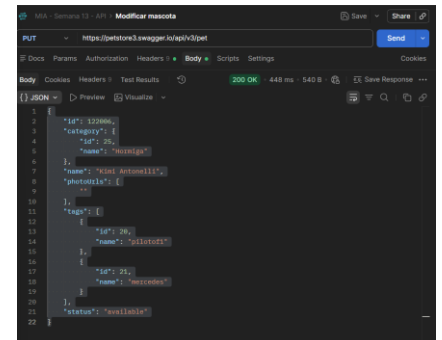
Flujo de Postman



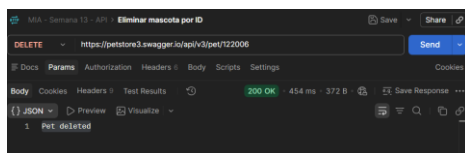
1. Registrar mascota



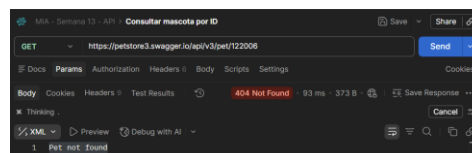
2. Consultar mascota



3. Actualizar estado



4. Eliminar mascota



5. Comprobar inexistencia

Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab1/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.